

CSIR11 – Introduction to Computer Programming

C language

by

Dr. Aswani Devi Aguru

Assistant Professor, Department of CA,

NIT Trichy



Course Learning Objectives

- ❖ CLO1 — Learn the fundamentals of computers.
- ❖ CLO2 — Learn problem-solving techniques, algorithms and procedures.
- ❖ CLO3 — Develop C code for simple logic using syntax and semantics.
- ❖ CLO4 — Understand structured programming: selection and iteration.
- ❖ CLO5 — Learn functions and file-handling techniques.

Course Outcomes & Assessments

- ❖ CO1 — Write algorithms for problems.
- ❖ CO2 — Demonstrate knowledge of C syntax and semantics.
- ❖ CO3 — Translate a given logic into correct C code.
- ❖ CO4 — Use C to solve computational problems.
- ❖ CO5 — Handle file structures and file I/O.

Assessments

Sl. No.	Mode of Assessment	Week / Date	Duration	% Weightage
1	1 st assessment	As per schedule	1 hr	15
2	2 nd assessment	As per schedule	1 hr	15
CPA*	Compensation Assessment	As per schedule	1 hr	15
3	Lab Evaluation	Every week	-	10
4	External Lab Assessment	As per schedule	2 hrs	20
5	End semester	As per schedule	3 hours	40

References

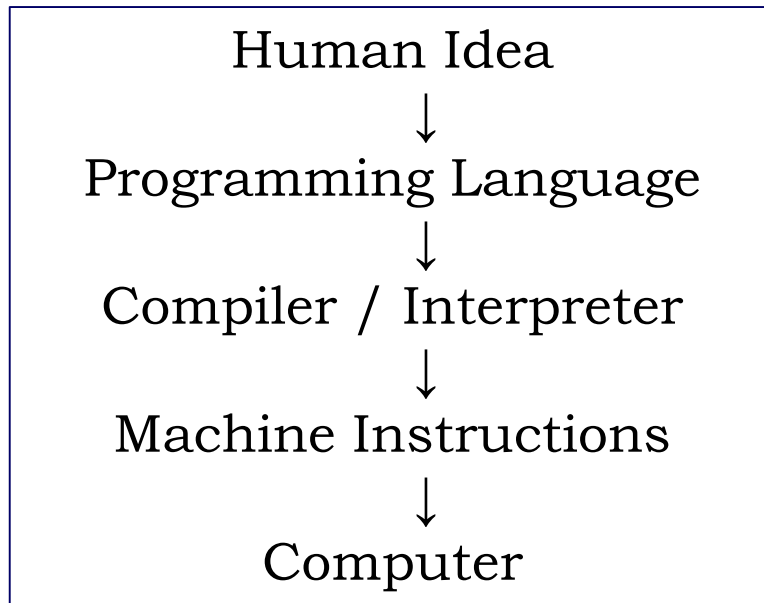
1. Byron Gottfried, Programming with C, 3rd Edition, Tata McGraw Hill Education, 2010. **[Introductory C]**
2. Brian W Kernighan and Dennis Ritchie, The C Programming language, 2nd Edition, Prentice Hall, 1988. **[Core C concepts]**
3. R.G. Dromey, How to solve it by Computers? Prentice Hall, 2011. **[Problem-solving]**
4. J.R.Hanly and E.B. Koffmann, Problem Solving and Program design in C, 6th Edition, Pearson Education, 2009. **[Program design]**
5. Paul Deital and Harvey Deital, C How to Program? 7 th Edition, Prentice Hall, 2012.
6. YashvantKanetkar, Let Us C, 12th Edition, BPB Publications, 2012.

Course Orientation

- ❖ **What Is Programming?**
- ❖ Programming is the process of giving precise sequence of instructions to a computer to solve a problem. [Ex: Calculate average of three numbers - algorithmic thinking]
- ❖ **A program tells the computer:**
 - ❖ What data to take as input
 - ❖ What operations to perform
 - ❖ What decisions to make
 - ❖ What repetition is required
 - ❖ What result to produce

Course Orientation (2)

- ❖ **Why Do We Need Programming Languages?**
- ❖ Computers ultimately execute machine instructions.
- ❖ Humans prefer understandable representations.



Abstraction between humans and hardware:
“Programming is the process of expressing a solution to a problem in a precise form that a computer can execute.”

Why Do Engineers Need Programming?

Engineering Branch	Programming Applications
ECE	Embedded systems, IoT, DSP, communication, microcontrollers
ICE	Sensors, data acquisition, PID control, automation, real-time systems
Mechanical	Robotics, CNC, mechatronics, automation, simulation
Electrical	Power systems, control, embedded systems, signal processing
Civil	Numerical analysis, structural computation, simulation, data processing

Programming Is NOT Just Coding

- ❖ **Problem** → **Immediately write C code**
- ❖ **Problem** → **Understand** → **Decompose** → **Algorithm** → **Test** → **Code** → **Debug**
- ❖ The algorithm is the bridge between the engineering problem and the programming language.
- ❖ **Everyday Example:** Making Tea
- ❖ Three Fundamental Building Blocks
 1. Sequence: Instructions execute one after another.
 2. Selection: Choose between alternatives.
 3. Iteration: Repeat instructions.

Why C?

- ❖ Small core language with powerful operators and control structures.
- ❖ Direct access to memory through pointers.
- ❖ Efficient compiled code and close relationship with hardware.
- ❖ Portable standard language used in systems, embedded and performance-oriented software.
- ❖ Requires discipline: memory, types, bounds and lifetime must be handled carefully.

**Procedural, Structured, Compiled, Efficient, Portable, Close to hardware,
Widely used in embedded systems**

Unit-1

Course Unitization Plan – Unit-1

1. Introduction to computers - Types of programming languages- Developing a program
2. Algorithms- Characteristics- Flow Charts- Principles of structured programming- Sequential
3. selecting structures- Repetitive Structures-Bounded, Unbounded and Infinite iterations.

What Is a Computer?

- ❖ A computer is an electronic programmable system that:
 - ❖ Accepts data as **input** (**Keyboard, Sensor, Camera**)
 - ❖ **Processes** the input according to instructions (**CPU, ALU, CU**)
 - ❖ Produces a meaningful **output** (**Monitor, Actuator, Printer**)
 - ❖ **Stores** data and results (**Primary memory, Secondary storage**)
- ❖ **CPU consists mainly of:**
 - ❖ ALU — Arithmetic Logic Unit: Addition, Subtraction, Comparisons, Logical operations
 - ❖ Control Unit: Coordinates operations, Fetches and interprets instructions
 - ❖ Registers: Very fast temporary storage

Key Functions:

- ❖ Input
- ❖ Processing
- ❖ Output
- ❖ Storage

Computer Organization

- ❖ Computer organization describes the operational units and their interconnections.
- ❖ **Major Components:**
 - ❖ Input Unit: Keyboard, mouse, scanner
 - ❖ Central Processing Unit (CPU):
 - ❖ Arithmetic Logic Unit (ALU)
 - ❖ Control Unit (CU)
 - ❖ Memory Unit: Primary and Secondary memory
 - ❖ Output Unit: Monitor, printer

Characteristics of Computer

- ❖ High speed
- ❖ Accuracy
- ❖ Automation
- ❖ Diligence/ Consistency
- ❖ Versatility
- ❖ Large storage capacity
- ❖ **Limitations:**
 - ❖ No intelligence or emotions
 - ❖ Works only as per instructions

Hardware vs Software

Hardware

- ❖ Physical components of a computer system
- ❖ CPU
- ❖ Memory (RAM, Hard disk)
- ❖ I/O devices (Keyboard, Sensors, Display)

Software

- ❖ Set of Programs/instructions that tell hardware what to do.
- ❖ **Types of Software:**
 - ❖ System Software (OS, Compilers)
 - ❖ Application Software
 - ❖ Embedded programs

Hardware provides capability; software provides instructions.

Computer System: Big Picture

- ❖ Input devices provide data; output devices communicate results.
- ❖ CPU executes instructions and coordinates operations.
- ❖ Main memory stores programs and data currently in use.
- ❖ Secondary storage preserves data across power cycles.
- ❖ Operating system manages hardware and provides services to programs.
- ❖ A compiler translates C source into machine-oriented code that can execute on the target system.

Modes of Operation

- ❖ **Batch Processing:** A large number of similar jobs are collected, grouped (batched), and executed together without user interaction. (**Use Cases:** Payroll Processing in an Organization)
- ❖ **Time Sharing:** Multiple users share the same computer system simultaneously. (**Use Cases:** Multi-User Computer Lab System)
- ❖ **Real-time Processing:** The computer processes data instantly and produces output within a strict time limit for immediate response. (**Use Cases:** Air craft positioning)
- ❖ **Distributed Processing:** Processing is distributed across multiple computers connected through a network, rather than being performed on a single system. (**Use Cases:** Cloud-Based Big Data Analytics)

Types of programming languages

- ❖ **Programming Languages:** A programming language is a formal language used to communicate instructions to a computer.
- ❖ **Types of Programming Languages:**
 - ❖ **Machine Language:** lowest-level programming language which CPU can understand; binary digits (0s and 1s)
 - ❖ **Assembly Language:** low-level programming language; mnemonic codes (ADD, MOV, SUB); An assembler converts assembly language into machine code.
 - ❖ **High-level Languages** (Python, C, Java): easy for humans to read, write, and understand; English-like syntax; compiler or interpreter converts them into machine code.

Programming Language Categories

1. **Procedural:** Focus on steps/procedures: C, Fortran
2. **Object-Oriented:** Focus on objects and classes: C++Java
3. **Functional:** Focus on functions and transformations: Haskell, Lisp
4. **Scripting / High-Level:** Python, JavaScript

Developing a program

❖ Steps involved in program development:

1. Problem Definition
2. Algorithm Design
3. Flowchart Representation
4. Coding
5. Testing and Debugging
6. Documentation and Maintenance

From Source Code to Execution

1. Edit — write source files such as program.c.
 2. Preprocess — handle directives such as #include and #define.
 3. Compile — translate C source into lower-level representation.
 4. Link — combine object code with required libraries.
 5. Load & execute — operating system prepares the program for execution.
- Debugging can involve any of these stages.

Program Development Process

1. Define the problem
2. Analyze the problem
3. Design the solution
4. Write algorithm
5. Draw flowchart / pseudocode
6. Write program
7. Compile
8. Test
9. Debug
10. Document & maintain

First C Program

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

Algorithm

❖ An algorithm is a finite sequence of well-defined instructions to solve a problem.

Characteristics of Algorithms:

1. Input: Zero or more inputs.
2. Output: At least one meaningful result.
3. Definiteness: Every step should be unambiguous.
4. Finiteness: It should terminate after a finite number of steps.
5. Effectiveness: Each step should be practical and executable.
6. Correctness: It should produce the expected result.

Algorithm Design Example - 1

❖ Example: Finding the Sum of Two Numbers

1. Start
2. Read two numbers A and B
3. Compute $S = A + B$
4. Display S
5. Stop

Algorithm Design Example - 2

❖ Example: Algorithm to Check Whether a Number is Even or Odd

1. Step 1: Start
2. Step 2: Read a number N
3. Step 3: If $N \bmod 2 = 0$, print “Even” (The mod operator (%) gives the remainder when one number is divided by another)
4. Step 4: Else print “Odd”
5. Step 5: Stop

$$25 \% 10 = ?$$

$$56 \% 7 = ?$$

$$1 \% 5 = ?$$

$$100 \% 9 = ?$$

Algorithm Design Example - 3

❖ Example: Algorithm to Find the Average of Two Numbers

❖ Algorithm:

1. Start
2. Read two numbers a and b
3. Calculate average = $(a + b) / 2$
4. Display the average
5. Stop

25 % 10 = ?
56 % 7 = ?
1 % 5 = ?
100 % 9 = ?

Algorithm Design Example - 4

Problem: Find the larger of two numbers.

Algorithm:

1. Read A and B
2. Compare A and B
3. If $A > B$, display A
4. Otherwise display B
5. Stop

Algorithm Design Example - 5

Problem: Find the Maximum of Three Numbers

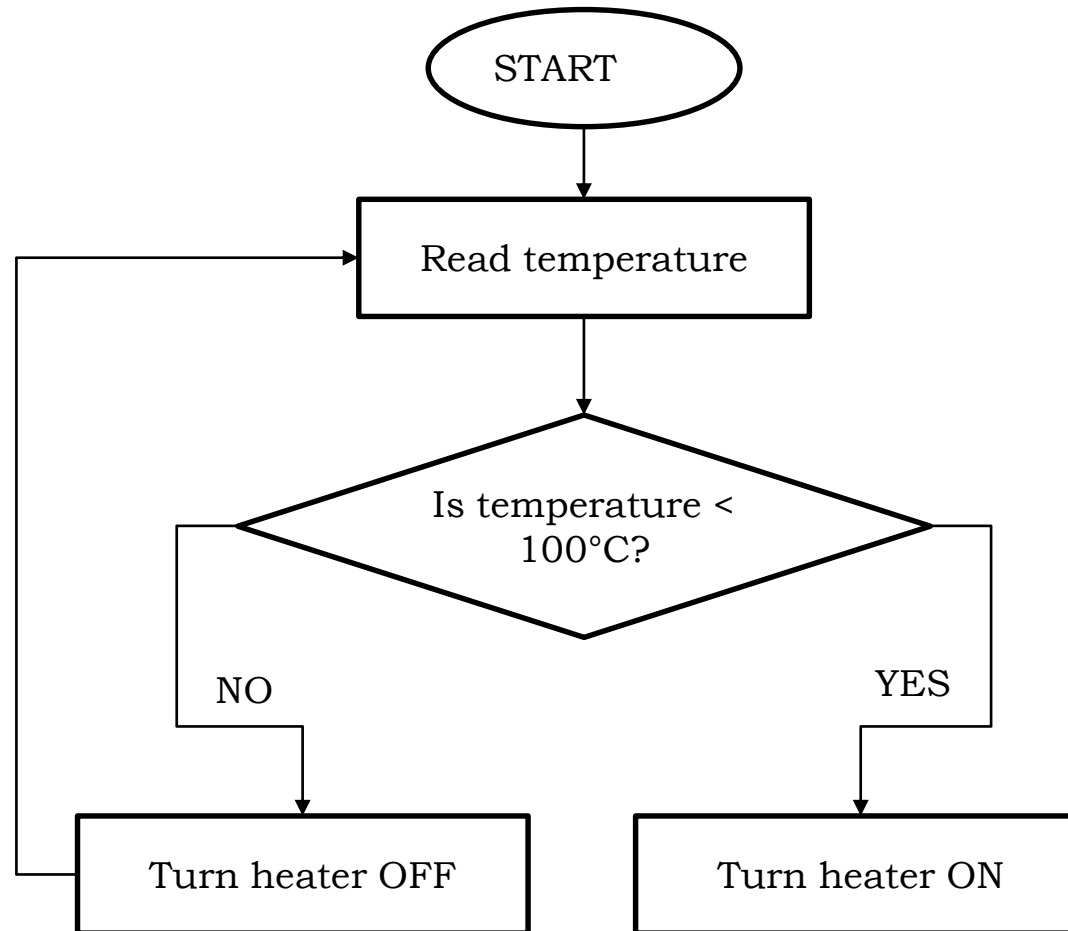
Algorithm:

1. Start
2. Read a, b, and c
3. Compare a and b
4. If $a > b$, compare a with c
5. Otherwise, compare b with c
6. Display the maximum
7. Stop

Flowcharts

- ❖ A flowchart is a graphical representation of an algorithm using symbols.
- ❖ **Common Flowchart Symbols:**
 - ❖ Oval – Start/Stop
 - ❖ Parallelogram – Input/Output
 - ❖ Rectangle – Processing
 - ❖ Diamond – Decision
 - ❖ Arrow – Flow direction
 - ❖ Circle – Connector
- ❖ **Advantages:** Easy to understand and Improves logical thinking

Algorithm: Control the temperature of a furnace (Desired temperature = 100°C)



Algorithm → C Program

Read two numbers
Calculate their sum
Display the result

```
#include <stdio.h>

int main(void)

{

    int a, b, sum;

    scanf("%d %d", &a, &b);

    sum = a + b;

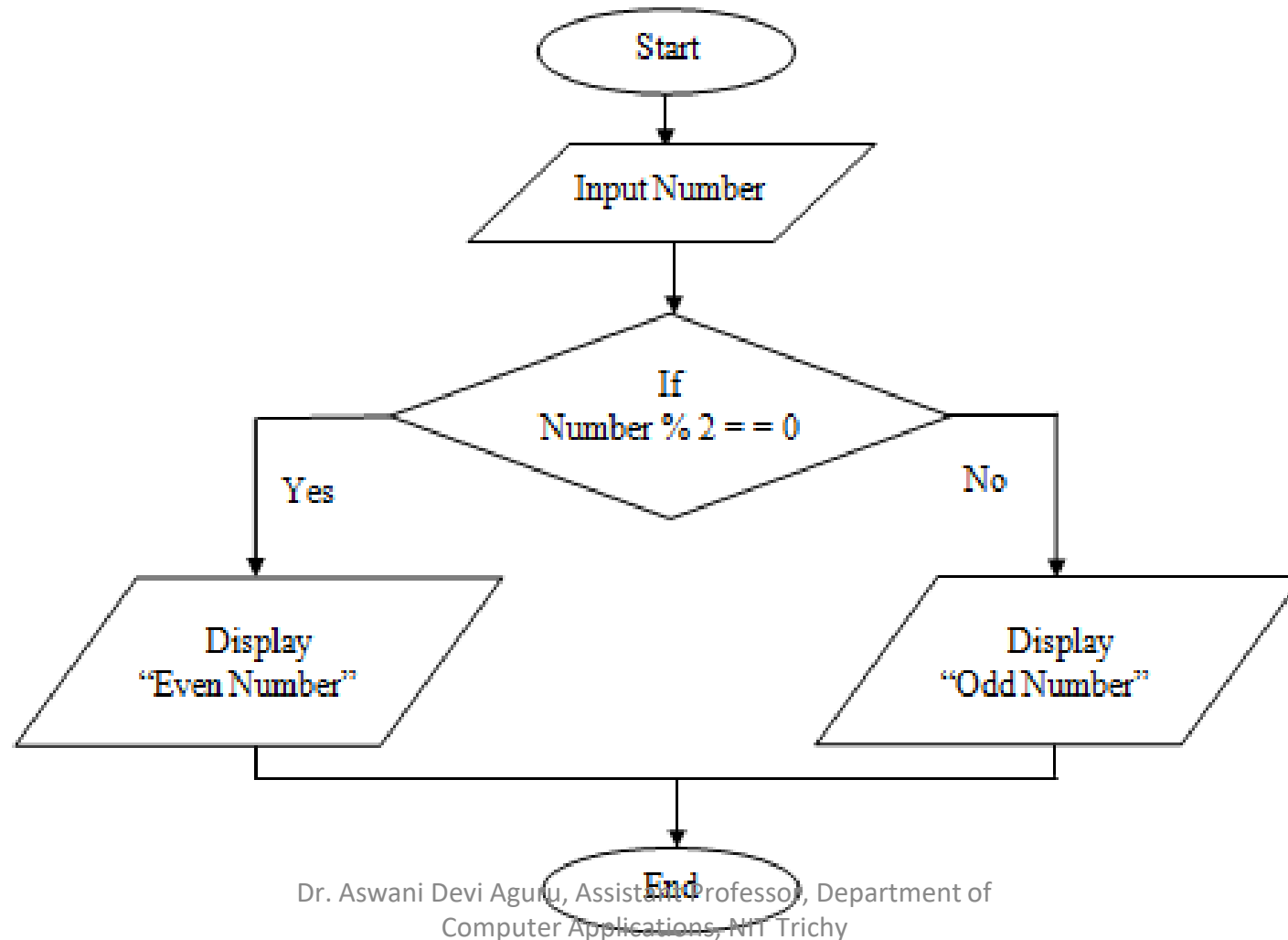
    printf("Sum = %d\n", sum);

    return 0;

}
```

Flowchart Example

- ❖ Flowchart for checking whether a number is even or odd.



Algorithm Example - Task

Algorithm to Swap Two Numbers (Using Temporary Variable)

Algorithm Example

❖ Example: Algorithm to Swap Two Numbers (Using Temporary Variable)

1. Step 1: Start
2. Step 2: Read two numbers A and B
3. Step 3: $\text{Temp} = A$
4. Step 4: $A = B$
5. Step 5: $B = \text{Temp}$
6. Step 6: Display A and B
7. Step 7: Stop

Algorithm Example – Task 2

Algorithm to Swap Two Numbers (Without Using a Temporary Variable)

Algorithm Example

❖ Example: Algorithm to Swap Two Numbers (Without Using a Temporary Variable)

1. Step 1: Start
2. Step 2: Read two numbers A and B
3. Step 3: $A = A + B$
4. Step 4: $B = A - B$
5. Step 5: $A = A - B$
6. Step 6: Display A and B
7. Step 7: Stop

Structured Programming

- ❖ Structured programming is a programming approach in which a program is organized using well-defined control structures.
- ❖ The three fundamental structures are:
 1. Sequence
 2. Selection
 3. Iteration
- ❖ Key principle: Break a complex problem into smaller, logically organized components.

Why Structured Programming?

1. Easier to understand
 2. Easier to test
 3. Easier to debug
 4. Easier to modify
 5. Easier to maintain
- ❖ **Sequential Structure:** Instructions execute one after another.
 - ❖ **Selection Structure:** To choose between alternatives.
 - ❖ **Repetitive Structure:** Iteration means repeatedly executing a set of instructions.

Selection Structures: Types of Selection

1. Single selection: if
2. Two-way selection: if ... Else
3. Multiple selection: if ... else if ... Else
4. Multiway selection: switch

```
if (marks >= 50)
    printf("Pass");
```

```
if (n > 0)
    printf("Positive");
else
    printf("Non-positive");
```

Repetitive Structure

Three Types of Loops in C

1. While: Condition checked before execution.
2. do-while: Condition checked after execution.
3. For: Commonly used when iteration is controlled by a counter.

What Is Bounded Iteration?

❖ The number of iterations is known or can be determined beforehand.

❖ Example: Print numbers from 1 to 10.

```
for (i = 1; i <= 10; i++)  
    printf("%d\n", i);
```

❖ Problem: Calculate the sum of first N natural numbers. Algorithm is:

1. Read N
2. SUM = 0
3. i = 1
4. Repeat while i <= N
5. SUM = SUM + i
6. i = i + 1
7. Display SUM

What Is Unbounded Iteration?

- ❖ The exact number of iterations is not known beforehand.
- ❖ The loop continues until a condition becomes false or a terminating event occurs.

Read sensor value

While sensor value is within operating range

Process value

Read next value

- ❖ We do not know beforehand how many readings will be processed.
- ❖ The number of iterations depends on the input.

What Is Infinite Iteration?

- ❖ An infinite loop continues indefinitely because its termination condition is never satisfied.

```
while (1)
{
    printf("Running...\n");
}
```

- Is an infinite loop always an error? - No.
- Infinite loops can be intentional in:
- Embedded systems
- Operating systems
- Servers
- Real-time controllers

Embedded Systems Example

- ❖ A microcontroller may continuously monitor a sensor.
- ❖ The system is designed to operate continuously.

```
while (1)
{
    temperature = read_sensor();

    if (temperature > LIMIT)
        turn_on_alarm();
}
```

Type	Iterations known beforehand?	Example
Bounded	Yes / predetermined	Process 100 measurements
Unbounded	No	Read until -1
Infinite	Never terminates normally	Embedded monitoring system

Algorithm: Control the temperature of a furnace (Desired temperature = 100°C)

1. Sequence

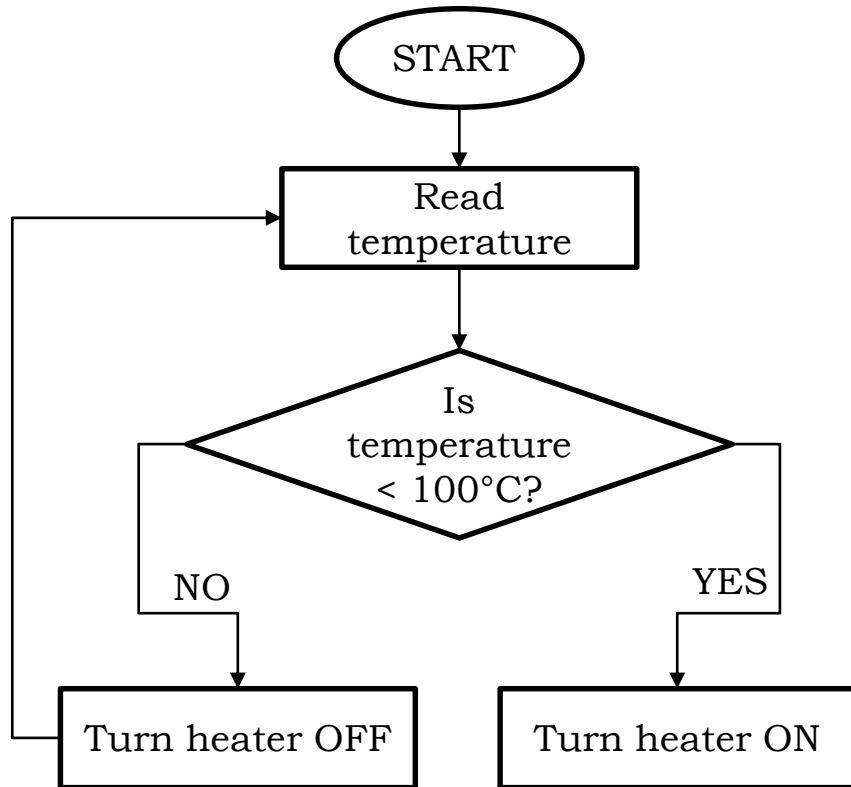
Read a
Read b
Calculate sum
Display sum

2. Selection

If temperature > limit
Turn OFF heater
Else
Keep heater ON

3. Iteration

While temperature is not stable
Read temperature
Adjust heater



Unit-2

Course Unitization Plan – Unit-2

1. Introduction to C- C character set- Identifiers and Keywords- Data types- Constants- Variables
2. Declarations- Expressions- Statements- Symbolic Constants- Operators- Library Functions
3. Data input and output: Single character input and output- Entering input data- Writing output
4. data- gets and puts functions - Control Statements- Branching: if-else-looping: while- do
5. While, for; Nested control Structures- switch statements- Break statements- Continue Statements
6. Comma operator- goto statements.

Introduction to C

- ❖ C is a general-purpose, procedural programming language developed in the early 1970s.
- ❖ C provides:
 1. Simple and expressive syntax
 2. Structured programming constructs
 3. Efficient execution
 4. Low-level memory access
 5. Portability
 6. Extensive library support

C helps us understand both programming concepts and how software interacts with computer hardware.

Basic Structure of a C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // Declare variables
```

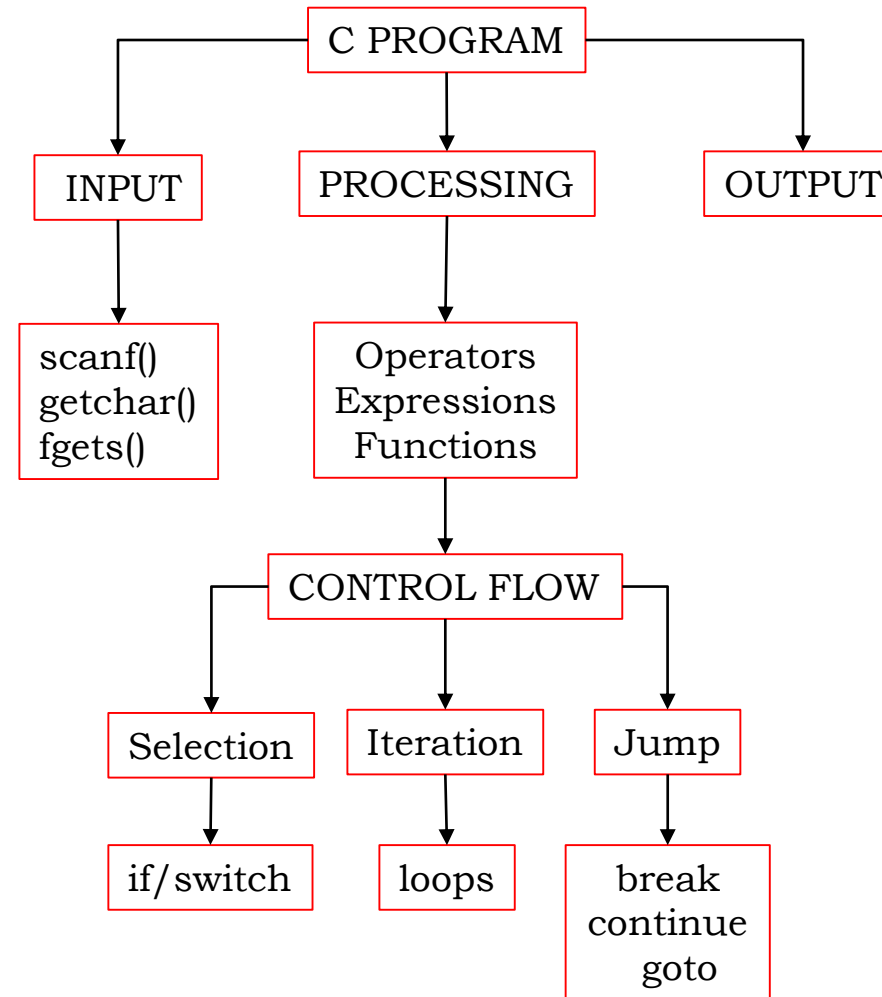
```
    // Take input
```

```
    // Process / calculate
```

```
    // Display output
```

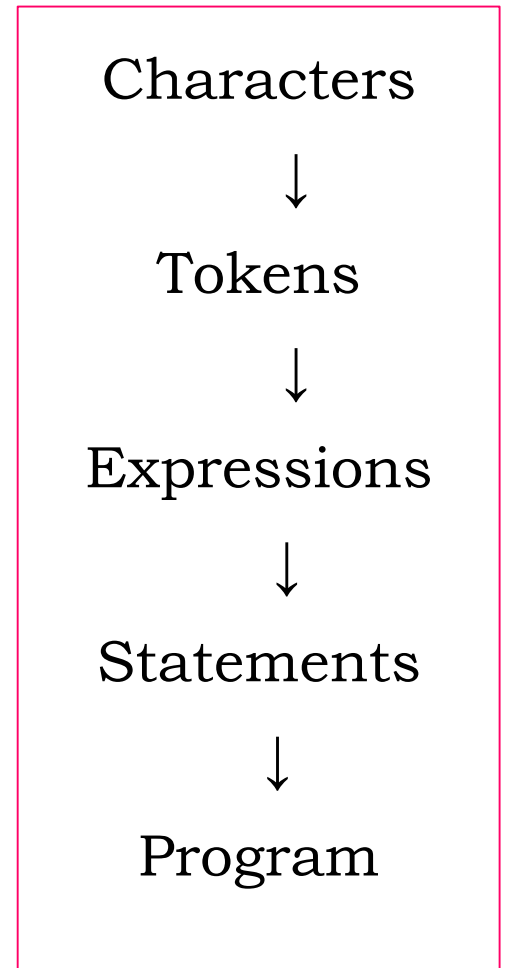
```
    return 0;
```

```
}
```



C character set

1. The C character set consists of characters that can be used to write C programs.
2. Main categories:
 - 1) Letters: A–Za–z
 - 2) Digits: 0–9
 - 3) Special characters: + - * / % = < > & | ! ; , . # () { } []
 - 4) Whitespace: Space, Tab, Newline



Identifiers

1. An identifier is a **name given by the programmer** to program elements such as:
Variables, Functions, Arrays, Structures
2. An identifier Can contain: Letters, Digits, Underscore _
3. **Rules:**
 - 1) Must not begin with a digit.
 - 2) Cannot contain spaces.
 - 3) Cannot be a C keyword.
 - 4) C is case-sensitive.

```
int age;  
float temperature;  
char grade;  
int total_marks();  
int ece_a[];
```

Keywords

Keywords are reserved words with predefined meanings in C. They cannot normally be used as programmer-defined identifiers.

TASK: Write the c key words and their usage?

- | | |
|-----------|--------------|
| 1. int | 9. switch |
| 2. float | 10. case |
| 3. char | 11. break |
| 4. if | 12. continue |
| 5. else | 13. struct |
| 6. for | 14. typedef |
| 7. while | 15. void |
| 8. return | |

Data types

C uses data types to specify the kind of data.

Type	Typical use	Example
char	Character	'A'
int	Integer	25
float	Single-precision real	3.14f
double	Double-precision real	3.14159
void	No value	void function()

Types of Data Types

1. Integer: Stores whole numbers.
2. Floating point: Stores numbers with fractional parts.
3. Character constants use single quotes.

```
int count = 25;
```

```
float temperature = 36.5f;  
double pi = 3.141592653589793;
```

```
'A'    // character  
"A"    // string
```

Variables

1. A variable is a named memory location whose stored value can change during program execution.

```
int age = 18;
```

Here:

- age → variable name
- 18 → initial value
- int → data type

Declarations

1. Variable Declaration: `data_type variable_name;`
2. Multiple declarations: `int a, b, c;`
3. A variable can be initialized when declared.

```
int x;      // declaration  
x = 10;     // assignment  
int x = 10; // declaration + initialization
```

Constants & Symbolic Constants

1. A constant is a value that does not change during the intended execution of the program.
2. Symbolic Constants: Instead of repeatedly writing a fixed value:

```
area = 3.14159 * r * r;
```

3. Advantage: Meaningful + maintainable + less repetition

Expressions

1. An expression combines operands and operators to produce a value.
2. Expression Evaluation is based on operator precedence.
3. Use Parentheses for Clarity, Instead of relying heavily on precedence

$a + b$
 $x * y$
 $\text{marks} \geq 40$
 $(a + b) / 2.0$
 $a + (b * c)$

Statements

1. A statement is an instruction that performs an action.

```
x = 10;
```

```
printf("Hello");
```

```
scanf("%d", &x);
```

```
x++;
```

```
return 0;
```

```
a + b // expression
```

```
sum = a + b; // statement containing an expression
```

Operators

Arithmetic Operators

+ Addition
- Subtraction
* Multiplication
/ Division
% Remainder

Logical Operators

&& AND
|| OR
! NOT

Conditional (Ternary) Operators

?:

Special Operators

sizeof → Size of data type/object
& → Address of variable
* → Dereference pointer
, → Comma operator
. → Structure member
-> → Structure member through pointer
[] → Array element
() → Function call
(type) → Type conversion

Assignment Operators

= += -= *= /= %=

Relational Operators

> < >= <= == !=

Bitwise Operators

& | ^ ~ << >>

Increment/decrement Operators

++ --

Library Functions

1. Programmers frequently need common operations. Instead of implementing everything from scratch, C provides library functions.
2. Library functions are associated with header files.
3. `#include <stdio.h>` provides declarations for standard input/output functions such as:

```
printf()  
scanf()  
getchar()  
putchar()  
fgets()  
puts()
```

```
sqrt()  
pow()  
strlen()  
malloc()  
fopen()
```

Standard Library Functions

C's standard library provides reusable functions.

stdio.h — input/output functions.

math.h — mathematical functions such as sqrt and pow.

string.h — string operations.

stdlib.h — dynamic memory, conversions, utilities.

Always include the appropriate header and use the documented function contract.

How Does a C Program Interact with the User?

1. Input provides data -> Processing manipulates data -> Output communicates results.

Format Specifiers

```
int age = 18;  
printf("%d", age);  
//Format Specifier → Variable
```

Data type	printf() format
int	%d
char	%c
float	%f
double	%lf
string	%s

Format Specifiers

```
int age = 18;
```

```
float temperature = 36.5f;
```

```
char grade = 'A';
```

```
printf("%d\n", age);
```

```
printf("%f\n", temperature);
```

```
printf("%c\n", grade);
```

Escape Sequences

Special characters can be represented using escape sequences.

Escape sequence	Meaning
\n	New line
\t	Horizontal tab
\\	Backslash
\"	Double quotation mark
\'	Single quotation mark

INPUT USING scanf()

- ❖ scanf() is used to read formatted input.
- ❖ scanf() needs the address where it should store the input. (using &)
- ❖ scanf() enables reading multiple values

```
#include <stdio.h>

int main(void)
{
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

Write a C program to read two numbers from the user and find their sum.

```
#include <stdio.h>

int main(void)
{
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

This tells the compiler to include the **standard input/output library**. We need it because we are using: printf() and scanf()

Every C program starts execution from the main() function. Think of main() as the starting point of the program.

We need memory to store data.

This displays a message to the user.

Input: takes two integers from the user.

This is the actual computation.

Output

It indicates that the program has completed successfully.

Reading Different Datatypes & Multiple values

```
int age;  
float salary;  
char grade;  
  
scanf("%d", &age);  
scanf("%f", &salary);  
scanf("%c", &grade);
```

```
int a, b;  
  
scanf("%d %d", &a, &b);
```

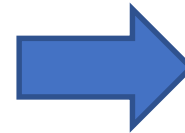
SINGLE CHARACTER I/O

getchar() //reads one character.

```
char ch;  
printf("Enter a character: ");  
ch = getchar();  
printf("You entered: %c", ch);
```

putchar() //displays one character.

```
char ch = 'A';  
putchar(ch);  
  
//Output is: A
```



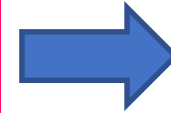
```
#include <stdio.h>
```

```
int main(void)  
{  
    char ch;  
  
    printf("Enter a character: ");  
    ch = getchar();  
    printf("You entered: ");  
    putchar(ch);  
  
    return 0;  
}
```

STRING I/O (using printf() and scanf())

```
char ch = 'A';           // character
char name[] = "IIT";     // string (a sequence of characters)
```

```
#include<stdio.h>
#include <string.h>
int main(void)
{
    char name[20];
    printf("Enter your name:");
    scanf("%s", &name);
    printf("\nHi %s. Welcome!", name);
}
```

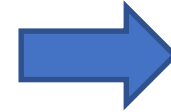


```
Enter your name:Aswani
Hi Aswani. Welcome!
```

Task: Replace "%s" with "%c", guess the output??

STRING I/O (using fgets() and puts())

```
#include<stdio.h>
#include <string.h>
int main(void)
{
    char name[20];
    puts("Enter your name:");
    fgets(name, sizeof(name), stdin);
    puts("Hi ");
    puts(name);
    puts("Welcome !");
}
```



Enter your name:Aswani

Hi

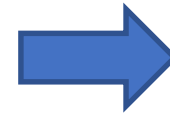
Aswani

Welcome !

Task: Print the output in single line using puts()..just like the previous slide?

STRING I/O (flexibility of fgets() and puts())

```
#include<stdio.h>
#include <string.h>
int main(void)
{
    char name[20];
    char message[50];
    puts("Enter your name:");
    fgets(name, sizeof(name), stdin);
    sprintf(message, sizeof(message), "Hi %s Welcome!", name);
    puts(message);
}
```



Enter your name:
Aswani

Hi aswani

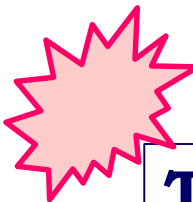
Welcome!

Task: But still...Welcome! Is in the next line ??

Exercise

Problem: Read a student's name and age and display them.

Enter name: aswani
Enter age: 28
Name = aswani
Age = 28



Task: use `fgets()` and `puts()` to read name, instead of `printf()` and `scanf()`



```
#include <stdio.h>
int main()
{
    char name[30];
    int age;
    printf("Enter name: ");
    scanf("%s", name);
    printf("Enter age: ");
    scanf("%d", &age);
    printf("Name = %s\n", name);
    printf("Age = %d\n", age);
    return 0;
}
```

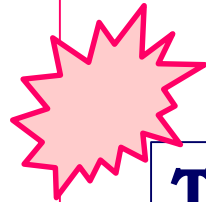


Program

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char name[30], output[50];
    int age;
    puts("Enter name:");
    fgets(name, sizeof(name), stdin);
    printf("Enter age: ");
    scanf("%d", &age);
    snprintf(output, sizeof(output), "Name = %s", name);
    puts(output);
    printf("Age = %d", age);
    return 0;
}
```

Enter name:
aswani
Enter age: 28
Name = aswani

Age = 28



Task: Remove
newline between
Name and Age
from output

CONTROL STATEMENTS

- ❖ CONTROL FLOW
 - ❖ Selection (if/else, switch)
 - ❖ Iteration (loops)
 - ❖ Jump (break, continue, goto)

Sequential Execution

❖ Statements normally execute from top to bottom.

❖ Why Do We Need Decisions?

If marks $\geq$ 50:
 Pass
Otherwise:
 Fail

```
int a, b, sum;  
  
scanf("%d %d", &a, &b);  
  
sum = a + b;  
  
printf("%d", sum);
```

**sequential
program**

❖ A sequential program alone cannot make this decision. So, we need selection.

if Statement: Syntax

```
if (condition)
{
    statement;
}
```

```
if (age >= 18)
{
    printf("Eligible to vote");
}
```

```
int n;

scanf("%d", &n);

if (n > 0)
{
    printf("Positive");
}
```

>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
==	Equal to
!=	Not equal

=	→	assignment
==	→	comparison

If-else Statement: Syntax

```
if (condition)
{
    statement1;
}
else
{
    statement2;
}
```

```
if (a > b)
{
    printf("Maximum = %d", a);
}
else
{
    printf("Maximum = %d", b);
}
```

```
if (n % 2 == 0)
{
    printf("Even");
}
else
{
    printf("Odd");
}
```

Else-if: Multiple Decisions

else-if Ladder

90–100 → A
80–89 → B
70–79 → C
60–69 → D
<60 → F



```
if (condition1)
{ //no
}
else if (condition2)
{//no
}
else if (condition3)
{//no
}
else
{//yes
}
```



```
if (marks >= 90)
    printf("A");
else if (marks >= 80)
    printf("B");
else if (marks >= 70)
    printf("C");
else
    printf("F");
```

Nested-if Statement: Syntax

```
if (condition1)
{
    if (condition2)
    {
        statement;
    }
}
```

```
if (n > 0)
{
    if (n % 2 == 0)
    {
        printf("Positive even");
    }
}
```

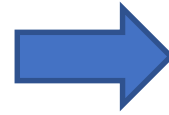
```
if (age >= 18)
{
    if (citizen == 1)
        printf("Eligible");
    else
        printf("Not eligible");
}
else
{
    printf("Underage");
}
```

else-if vs Nested if

else-if	Nested if
Multiple alternatives	Decision inside another decision
Conditions usually at same level	Conditions depend on outer condition
Useful for grading	Useful for dependent decisions

STRING I/O (advanced)

```
#include<stdio.h>
#include <string.h>
int main(void)
{
    char name[20];
    char message[50];
    puts("Enter your name:");
    fgets(name, sizeof(name), stdin);
    size_t len = strlen(name);
    if (len > 0 && name[len - 1] == '\n') {
        name[len - 1] = '\0'; }
    snprintf(message, sizeof(message), "Hi %s Welcome!", name);
    puts(message);
}
```



Enter your name:
aswani
Hi aswani Welcome!

Program

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char name[30], output[50];
    int age;
    puts("Enter name:");
    fgets(name, sizeof(name), stdin);
    size_t len = strlen(name);
    if (len > 0 && name[len - 1] == '\n') { name[len - 1] = '\0';}
    printf("Enter age: ");
    scanf("%d", &age);
    snprintf(output, sizeof(output), "Name = %s", name);
    puts(output);
    printf("Age = %d\n", age);
}
```

Enter name:
aswani
Enter age: 28
Name = aswani
Age = 28

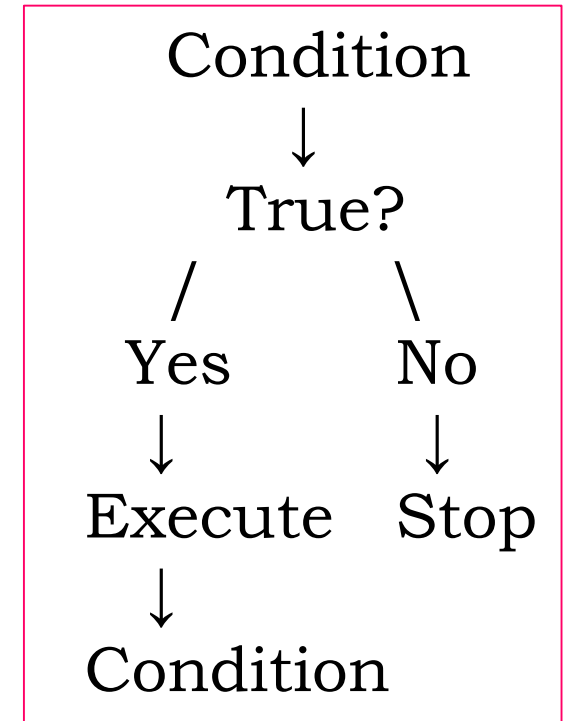
Looping: Repetition

❖ A loop repeatedly executes a block of statements.

❖ Why Loops?

❖ Print Hello 100 times

```
printf("Hello");  
printf("Hello");  
printf("Hello");  
...
```



❖ The comma , operator can separate multiple expressions.

1. **while loop** → checks the condition before each iteration.
2. **do-while loop** → runs at least once, checks condition after each iteration.
3. **for loop** → initialization, condition, and increment in one line.

Looping: Syntax

//while loop

```
while (condition)
{
    statements;
}
```

// do-while loop

```
do
{
    statements;
}
while (condition);
```

Unbounded Iteration

The number of repetitions is **not known in advance**.

// Using for loop

```
for (initialization; condition; update)
{
    statements;
}
```

Bounded Iteration

The number of repetitions is known or controlled by a clear limit.

Looping: Programs

// Using while loop

```
#include <stdio.h>
int main(void)
{
    int i;
    i = 1;
    while (i <= 100)
    {
        printf("Hello\n");
        i++;
    }
}
```

// Using do-while loop

```
#include <stdio.h>
int main(void)
{
    int i;
    i = 1;
    do
    {
        printf("Hello\n");
        i++;
    } while (i <= 100);
}
```

// Using for loop

```
#include <stdio.h>
int main(void)
{
    int i;
    for (i = 1; i <= 100; i++)
    {
        printf("Hello\n");
    }
}
```

While loop: Condition-controlled repetition

❖ Print 1 to 5

```
#include <stdio.h>
int main(void){

    int i = 1;
    while (i <= 5)
    {
        printf("%d ", i);
        i++;
    }
}
```

1 2 3 4 5

i	i <= 5	Output
1	True	1
2	True	2
3	True	3
4	True	4
5	True	5
6	False	Stop

DRY RUN

Do-while loop: Execute at least once

❖ Print 1 to 5

```
#include <stdio.h>
int main(void){
    int i = 1;
    do{
        printf("%d ", i);
        i++;
    }while (i <= 5);
}
```

1 2 3 4 5

i	Output	i <= 5
1	1	True
2	2	True
3	3	True
4	4	True
5	5	True
6	—	False
	Stop	

DRY RUN

For loop: Count-controlled repetition

❖ Print 1 to 5

```
#include <stdio.h>
int main(void){
    for (int i = 1; i <= 5; i++)
    {
        printf("%d ", i);
    }
}
```

1 2 3 4 5

i	i <= 5	Output	i++
1	True	1	2
2	True	2	3
3	True	3	4
4	True	4	5
5	True	5	6
6	False	—	Stop

DRY RUN

Infinite loops

```
while (1)
{
    printf("Hello");
}

//Infinite Loop
//Condition is always true
//Infinite repetition
```

```
for (;;)
{
    printf("Running...");
}

//Infinite Loop
//Condition is always true
//Infinite repetition
```

Nested Loops

❖ Printing a Pattern

```
for (...)  
{  
    for (...)  
    {  
        ...  
    }  
}
```

```
for (int i = 1; i <= 5; i++)  
{  
    for (int j = 1; j <= i; j++)  
    {  
        printf("* ");  
    }  
    printf("\n");  
}
```



```
*  
* *  
* * *  
* * * *  
* * * * *
```

Multi-Way Selection and Loop Control

- ❖ **Switch:** when one expression must be compared with several constant cases.
- ❖ **Break:** immediately terminates the loop. Without break, execution can continue into the next case. This is called fall-through.
- ❖ **Continue:** skips the current iteration and proceeds to the next iteration.

break	continue
Terminates loop	Skips current iteration
Execution leaves loop	Loop continues
Used to stop early	Used to ignore selected iteration

Multi-Way Selection and Loop Control

```
switch (expression)
{
    case value1:
        statements;
        break;

    case value2:
        statements;
        break;

    default:
        statements;
}
```

```
int choice;
scanf("%d", &choice);
switch (choice)
{
    case 1:
        printf("Add");
        break;
    case 2:
        printf("Subtract");
        break;
    default:
        printf("Invalid choice");
}
```

break and continue

```
for (int i = 1; i <= 10; i++)  
{  
    if (i == 5)  
        break;  
  
    printf("%d ", i);  
}
```

1 2 3 4

```
for (int i = 1; i <= 5; i++)  
{  
    if (i == 3)  
        continue;  
  
    printf("%d ", i);  
}
```

1 2 4 5

goto

- ❖ Transfers control to a labeled statement.

Useful in some low-level/error-handling situations.

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter 1 to continue: ");
    scanf("%d", &n);
    if (n != 1)
        goto stop;
    printf("Program continues...");
stop:
    printf("End");
    return 0;
}
```

Program Tracing

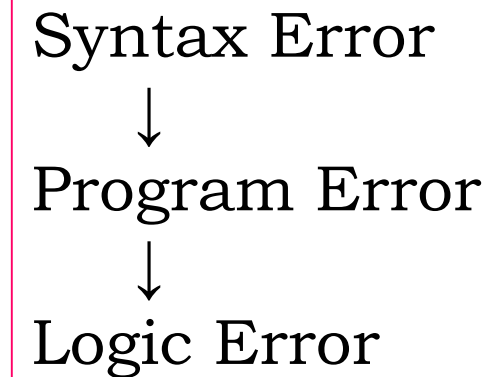
- ❖ Following the execution of a program step by step.
- ❖ Why Trace Programs?
- ❖ understand program execution
- ❖ find logical errors
- ❖ predict output
- ❖ understand loops
- ❖ understand nested conditions

```
int x = 5;  
int y = 2;  
  
x = x + y;  
y = x - y;
```

Step	x	y
Initial	5	2
x=x+y	7	2
y=x-y	7	5

Debugging

❖ Finding and fixing errors in a program.



❖ **Syntax Error:** Compiler reports an error.

❖ **Runtime Error:** Program compiles but encounters a problem during execution.

❖ **Logical Error:** Program runs but produces the wrong answer. The program may compile successfully, but the logic is wrong.

Errors

// Syntax Error

```
printf("Hello"
```

// Runtime Error

```
int a = 10;  
int b = 0;
```

```
printf("%d", a / b);
```

// Logical Error

```
average = a + b / 2;  
average = (a + b) / 2;
```

Thank You